

Explainability Methods for Hardware Trojan Detection: A Systematic Comparison

Paul Whitten^{1*}, Francis Wolff^{1†} and Chris Papachristou^{1†}

^{1*}Electrical, Computer, and Systems Engineering, Case Western Reserve University, 10900 Euclid Avenue, Cleveland, 44106, Ohio, USA.

*Corresponding author(s). E-mail(s): pcw@case.edu;
Contributing authors: fxw12@case.edu; cap2@case.edu;

[†]These authors contributed equally to this work.

Abstract

Hardware trojans are malicious circuits which compromise the functionality and security of an integrated circuit (IC). These circuits are manufactured directly into the silicon and cannot be fixed by security patches like software. The solution would require a costly product recall by replacing the IC and hence, early detection in the design process is essential. Hardware detection at best provides statistically based solutions with many false positives and false negatives. These detection methods require more thorough explainable analysis to filter out false indicators.

Existing explainability methods developed for general domains like image classification do not always provide the actionable insights hardware engineers need. A question remains: how do domain-aware property analysis, model-agnostic case-based reasoning, and model-agnostic feature attribution techniques compare for hardware security applications?

This work compares three categories of explainability for gate-level hardware trojan detection on the Trust-Hub benchmark dataset: (1) domain-aware property-based analysis of 31 circuit-specific features derived from gate fanin patterns, flip-flop distances, and primary Input/Output (I/O) connectivity; (2) model-agnostic case-based reasoning using k-nearest neighbors for precedent-based explanations; and (3) model-agnostic feature attribution methods (Local Interpretable Model-agnostic Explanations (LIME), SHapley Additive exPlanations (SHAP), gradient) that provide generic importance scores without circuit-level context.

The findings show that different explainability approaches offer distinct advantages for hardware security practitioners. The domain-aware property-based method analyzes 31 circuit properties. Detection decisions are explained through

familiar concepts like “high fanin complexity near outputs indicates potential trojan triggers.” Case-based reasoning (k-nearest neighbors) achieves 96.51% correspondence between predictions and training exemplars. In contrast, LIME and SHAP show only moderate per-gate agreement (Spearman $\rho = 0.30$ mean over $n = 11,392$ gates, 95% bootstrap CI $[0.29, 0.31]$; global concatenated $\rho = 0.31$, $p \ll 10^{-300}$), and yield generic feature importance scores that lack circuit-level context for validation or remediation planning. Detection performance using eXtreme Gradient Boosting (XGBoost) classification with optimized threshold achieves 48.08% precision and 69.44% recall on 11,392 held-out test gates (F1 = 0.568, Matthews Correlation Coefficient (MCC) = 0.575, area under the precision-recall curve (AUPRC) = 0.637) at the optimized threshold of 0.940. This represents a 4.25-fold precision improvement over a support vector machine (SVM) baseline reimplemented under identical experimental conditions (11.33% precision, 70.83% recall, F1 = 0.195 at threshold 0.050) with a 7.4-fold reduction in false-positive density (4.74 vs. 35.0 FP per 1,000 gates). Random Forest achieves comparable F1 (0.555, 58.46% precision, 52.78% recall) with half the false-positive density of XGBoost (2.37 vs. 4.74 FP per 1,000 gates), confirming that the explainable AI (XAI) findings generalize across classifier choices.

Additionally, gradient-based feature attribution (Simonyan et al., 2013), with a speedup factor of 7 over SHAP, yields the same model-agnostic feature weights as SHAP and LIME, confirming that computational efficiency alone cannot substitute for domain-aware interpretability.

This work provides empirical evidence comparing domain-aware property analysis, model-agnostic case-based reasoning, and model-agnostic feature attribution techniques for hardware security applications. The results show that property-based and case-based approaches offer complementary advantages: domain alignment and precedent-based interpretability, respectively, compared to generic feature rankings. These findings have implications for XAI deployment across domains where practitioners must validate and act on Machine Learning (ML) predictions.

Keywords: explainable artificial intelligence, hardware trojan detection, domain-aware explainability, model-agnostic explanations, circuit property analysis, LIME, SHAP, XGBoost, Trust-Hub benchmark, Case-explainer

1 Introduction

When a machine learning system flags a gate in a billion-transistor IC as a potential hardware trojan, the security engineer needs an explanation in circuit terms. A bare “feature importance: 0.73” does little to support validation or remediation. Without interpretations grounded in circuit-design principles, practitioners cannot separate real alerts from false positives, verify suspect structural patterns against domain knowledge, or translate detections into security actions.

Hardware trojans are malicious circuit modifications introduced during design or fabrication. Unlike software bugs, they are permanent in silicon, activate through rare input combinations engineered to evade conventional test (often 1 in 2^{64} states),

and operate at the gate level with no software-visible trace, potentially compromising confidentiality, integrity, or availability.

Machine learning models flag suspicious circuit structures from gate-level netlists, but opaque-box predictions lack the interpretable justifications hardware security engineers need. General-purpose XAI techniques such as LIME [1] and SHAP [2] explain predictions through feature perturbation and game-theoretic attribution; they are domain-agnostic and broadly applicable, but produce generic importance scores. Domain-aware methods, in contrast, encode circuit-specific knowledge (fanin patterns, flip-flop connectivity, proximity to primary I/O) directly into the explanation, aligning with how hardware engineers already reason about circuits but requiring domain effort to design. *Do model-agnostic explainability techniques developed for general ML applications provide the actionable insights hardware security practitioners need, or do domain-aware methods tailored to circuit characteristics offer better interpretability?* Prior work has not answered this empirically.

This work provides such an empirical comparison on the Trust-Hub benchmark, contrasting domain-specific circuit-property analysis with general-purpose feature attribution.

The primary contributions are:

1. **Domain-Aware Property Analysis:** A property-based explainability method that analyzes 31 gate-level circuit-specific features grounded in hardware design principles. It produces explanations such as, “High LGFi = 12 at 2 levels upstream combined with low FFi = 1 matches rare-event trigger patterns,” which engineers can validate using familiar circuit-analysis concepts.
2. **Case-Based Reasoning with k-Nearest Neighbors:** An inherently interpretable predictor ($k = 5$) achieves 96.51% correspondence between predictions and training exemplars, justifying detections by precedent: “this gate’s feature profile matches 4 of 5 similar training cases that were trojans.”
3. **Systematic Quantitative Comparison with Model-Agnostic Baselines:** M2–M5 use a single XGBoost classifier with five different post-hoc explanation backends, isolating the effect of the explanation mechanism from the underlying detector. LIME and SHAP show moderate per-gate agreement (Spearman $\rho = 0.30$ over $n = 11,392$ gates, 95% bootstrap CI $[0.29, 0.31]$; global concatenated $\rho = 0.31$, $p \ll 10^{-300}$): significantly correlated but far from identical. Gradient-based attribution [3] is also evaluated for computational efficiency.
4. **Improved Detection Performance with XGBoost and Random Forest:** XGBoost achieves 48.08% precision at 69.44% recall ($F1 = 0.568$, $MCC = 0.575$, $AUPRC = 0.637$); Random Forest achieves 58.46% precision at 52.78% recall ($F1 = 0.555$, $MCC = 0.553$, $AUPRC = 0.512$). Both substantially improve over the Hasegawa SVM baseline reimplemented under identical conditions (11.33% precision, 70.83% recall, $F1 = 0.195$): $4.25\times$ precision (XGBoost) and $7.4\times$ lower false-positive density (4.74 vs. 35.0 FP per 1,000 gates).

Section 2 covers hardware trojans and XAI background. Section 3 reviews related work. Sections 4 and 5 present methodology and experimental setup. Section 6 reports results. Section 7 discusses limitations and future work. Section 8 concludes.

2 Background

2.1 Hardware Trojans

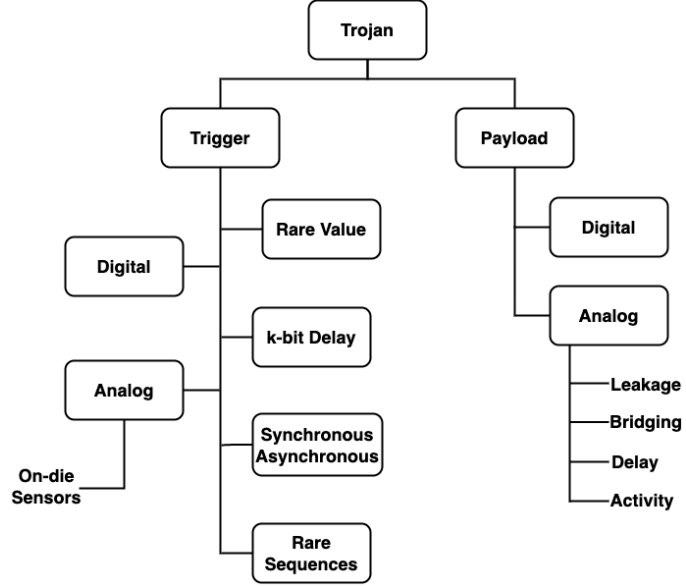


Fig. 1: Trojan circuit taxonomy[4].

Cost-driven outsourcing of integrated-circuit fabrication creates an opening for hardware trojans that weaken confidentiality, integrity, or availability of trusted ICs. Figure 1 shows the standard taxonomy: every trojan consists of a trigger and a payload, each of which may be digital or analog, with rare values, k -bit delays, or rare input sequences as typical activation conditions and control/status/data alteration or analog leakage as typical exploits [4].

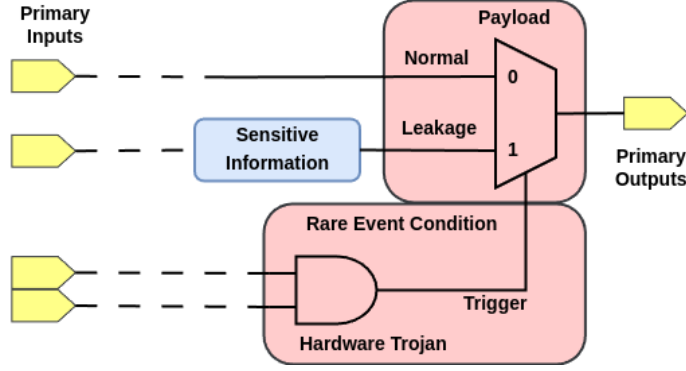


Fig. 2: Confidentiality rare event hardware trojan model[5].

Figure 2 shows a confidentiality trojan implemented in gate-level logic: a rare-event AND trigger conditioned on primary inputs gates a multiplexer payload that leaks sensitive data to a primary output. Detectability rests on the trigger being rare (e.g., one of 2^n input combinations) and the payload reaching an observable output, so static gate-level analysis can flag suspect subcircuits by fanin complexity and reachability to primary outputs [4, 6–8].

Hasegawa et al. [6, 7] encode this intuition as five per-net features used throughout this work: Logic Gate Fanins (LGF_i, inputs to logic gates two levels upstream), Flip-Flop Input (FF_i, upstream logic levels to a flip-flop), Flip-Flop Output (FF_o, downstream logic levels to a flip-flop), Primary Input (PI, upstream levels to the nearest primary input), and Primary Output (PO, downstream levels to the nearest primary output). Their original SVM addresses class imbalance via dynamic weighting; subsequent work uses synthetic oversampling [9] and other rebalancing strategies but does not improve the interpretability of the resulting decisions.

2.2 Trust-Hub Benchmark Dataset

The Trust-Hub benchmark [8, 10–12] provides gate-level netlists with labeled trojan insertions. This work uses 30 digital combinational circuits drawn from RS232 communication peripherals and ISCAS benchmarks; analog and transistor-level circuits are out of scope.

3 Related Work

3.1 Machine Learning for Hardware Trojan Detection

3.1.1 Hasegawa et al. Baseline

Hasegawa et al. established a foundational approach for ML-based hardware trojan detection from gate-level netlists [6, 7]. Their method extracts five structural features from each net: LGF_i, FF_i, FF_o, PI, and PO. Using Support Vector Machine (SVM)

classification with dynamic class weighting to address data imbalance, they reported 82.6% recall on the Trust-Hub benchmark dataset.

Precision computed from their published confusion matrices is 5.13%. Approximately 95% of gates flagged as trojans are false positives. Their published confusion matrices (Table V, dynamic weighting) show an average true negative rate of 49.5%, corresponding to a false positive rate of approximately 50.5%. The sheer volume of false positives limits practical utility for real-world deployment where security engineers must manually inspect falsely flagged gates.

To provide an apples-to-apples comparison, this work reimplements the Hasegawa SVM baseline under identical experimental conditions: the same Trust-Hub dataset, same five structural features, and same 60/20/20 random split. The reimplemented SVM (radial basis function (RBF) kernel, `class_weight='balanced'`) achieves 11.33% precision and 70.83% recall at the val-optimal threshold of 0.050 ($F1 = 0.195$, $MCC = 0.274$, 35.0 FP per 1,000 gates). XGBoost achieves 48.08% precision at threshold 0.940, a 4.25-fold precision improvement with a 7.4-fold reduction in false-positive density (4.74 vs. 35.0 FP per 1,000 gates). Random Forest further reduces the FP burden to 2.37 FP per 1,000 gates (14.8-fold below the SVM baseline) at 58.46% precision.

3.1.2 Recent Advances

Rathor and Rastogi’s HT-Pred system [13] uses 479 Trust-Hub circuits with 605 structural and functional features, substantially larger than prior published datasets. Their Deep Neural Network achieves 98.95% accuracy for circuit-level binary classification (trojan-infected vs. trojan-free circuits). However, their work addresses a fundamentally different problem than this work: they classify entire circuits rather than localizing specific trojan gates within netlists. Circuit-level classification benefits from balanced datasets (approximately 1:1 ratio of infected to clean circuits) and provides limited actionability, as security engineers still must manually search through hundreds or thousands of gates to locate the actual trojan components.

In contrast, this work performs gate-level (node-level) trojan localization on heavily imbalanced datasets (approximately 158 to one ratio of benign to trojan gates), directly identifying suspicious gates to enable targeted remediation. While HT-Pred achieves higher overall accuracy on the circuit-level task, the gate-level localization provides the specific information needed for practical trojan removal or mitigation.

3.1.3 Ensemble Methods for Hardware Trojan Detection

Negishi and Togawa [9] compare Random Forest, AdaBoost, and Gradient Boosting against SVM on Trust-Hub, finding ensemble methods consistently improve F-measure on the imbalanced detection task. Method 1 in this work uses an ensemble of 31 XGBoost classifiers, each trained on a distinct subset of the five Hasegawa features rather than on resampled data, aggregated by E_{PARS} -weighted voting. The subset structure yields property-level interpretability that homogeneous ensembles (Random Forest, AdaBoost) do not provide.

3.1.4 Class Imbalance Handling in Hardware Trojan Detection

Trust-Hub gate-level netlists carry a 158:1 benign-to-trojan ratio overall and 157:1 in the held-out test fold. Classifiers trained without imbalance handling bias toward the majority class. Standard responses include reweighting [14], synthetic oversampling, and cost-sensitive learning. This work uses XGBoost’s `scale_pos_weight` set to $N_{\text{benign}}/N_{\text{trojan}}$, following King and Zeng’s [14] weighting prescription, and achieves 69.44% recall on the 157:1 test fold.

Table 1 positions the present work against representative prior gate-level trojan detection methods on the Trust-Hub benchmark. Same-protocol rows use identical splits and thresholding; published rows are included for context and are not directly comparable.

The same-protocol rows establish the comparison of interest: under identical splits, thresholding, and metrics, the gradient-boosted detector substantially improves precision over the Hasegawa SVM baseline (48.08% vs. 11.33%, a 4.25-fold improvement) at comparable recall. Graph-based detectors operate on richer local features and are not directly comparable on this five-feature representation. Their published numbers are included to situate the line of work rather than to claim parity.

3.2 Explainable AI Techniques

3.2.1 General XAI Methods

Feature-importance methods identify which input features drive predictions, globally or locally; example-based methods such as case-based reasoning [17] retrieve similar training instances as explanations; attribution methods (LIME [1], SHAP [2]) assign per-feature importance scores, with SHAP offering local-accuracy and consistency guarantees at higher compute cost.

3.2.2 XAI for Hardware Security

Explainable AI applied to gate-level trojan detection remains thin. A systematic search over 2016–2025 publications (queries: “explainable AI hardware trojan,” “interpretable machine learning hardware security,” “XAI hardware trojan detection,” “explainability gate-level netlist analysis”) surfaces no prior work that systematically compares domain-aware against model-agnostic XAI methods on the Trust-Hub benchmark. Most ML-based trojan detection uses opaque-box models without explainability [6, 7, 9, 13]; when explanations appear they are typically Random-Forest feature-importance rankings without cross-method evaluation.

The most directly related XAI works: Pan and Mishra [18] apply SHAP and LIME to hardware-malware detection over performance-counter features, a different feature space, but the same XAI-auditing motivation. Mahfuz et al. (SALTY) [15] couple a graph neural network with post-hoc XAI refinement and report $\text{TPR} = 98.47\%$, $\text{TNR} = 98.14\%$ on a 125-dimensional local feature set under leave-one-family-out cross-validation, illustrating the headroom available from richer representations. Yasaei et al. [16] show that GNN feature representations improve cross-circuit generalization over the five-feature Hasegawa baseline. Ma et al. [19] extend GNN-based detection

Table 1: Gate-level trojan detection on the Trust-Hub benchmark across representative methods. “Same-protocol” rows use this work’s 60/20/20 random split, threshold selected on the validation set, and the 11,392-gate held-out test set. “Published” rows quote headline figures from the cited papers under each paper’s own protocol. These figures are not directly comparable to the same-protocol rows.

Method	Classifier	Protocol	P (%)	R (%)	F1	Source
Hasegawa et al. [6]	SVM (RBF)	Published (LOCO)	5.13	82.6	0.097	[6] Table V
Hasegawa et al. (reimpl.)	SVM (RBF)	Same-protocol	11.33	70.83	0.195	This work
This work	XGBoost	Same-protocol	48.08	69.44	0.568	This work
This work	RF	Same-protocol	58.46	52.78	0.555	This work
SALTY [15]	Graph attention	Published (LOFO)	— ^d	98.47 ^d	— ^d	[15] (125-dim graph features)
GNN4HT [16]	GNN	Published	—	—	—	No P/R/F1 reported [16]

^dSALTY reports TPR/TNR (98.47%/98.14%) on 125-dim local graph features under leave-one-family-out cross-validation; precision and F1 not reported. Published rows use different features and protocols and are not directly comparable to same-protocol rows. LOCO: leave-one-circuit-out cross-validation; LOFO: leave-one-family-out cross-validation.

with harmonic centrality on Trust-Hub. Su et al. [20] propose an explainable GNN for LUT-level trojan localization in FPGA netlists. None of these systematically compare XAI methods on a fixed gate-level structural feature representation.

This work fills that gap by comparing five explainability methods spanning domain-aware property analysis, model-agnostic case-based reasoning, and model-agnostic feature attribution (LIME, SHAP, gradient) on classification performance, explanation consistency, and computational efficiency.

3.3 Explainable Artificial Intelligence

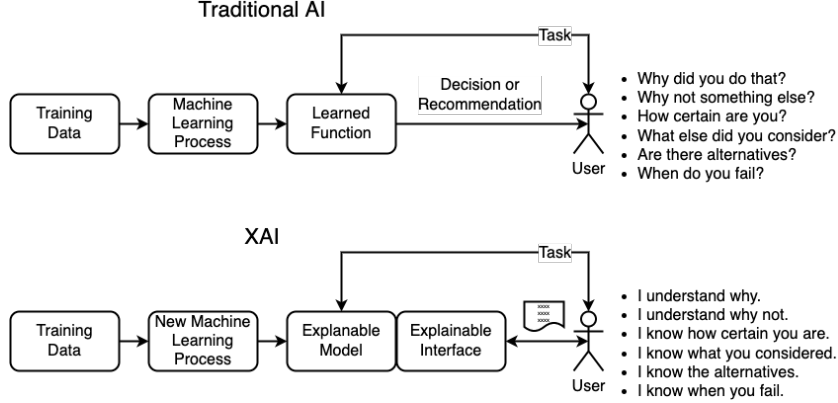


Fig. 3: Traditional AI interactions contrasted to XAI[21].

Figure 3 illustrates the goal for explainable artificial intelligence. The current ML interactions are depicted along the top as traditional AI. Because ML acts as an opaque box, users are left questioning the decision provided by the AI. The aim is to provide users with an explainable model and explainable interface. The resulting system, shown at the bottom of the figure, is able to present explanations along with a decision or recommendation. To improve trust, the explanations need to provide an understanding of the system’s overall reasoning and alternatives. The goal of XAI is to answer these important questions depicted in Figure 3 for users[21].

Mapping a learning classifier to human-comprehensible explanations remains challenging. Surveyed techniques span decision-tree, rule-based, salience-mapping, Grad-CAM, sensitivity analysis, feature-importance, fuzzy-neural, and genetic-programming families, and are conventionally evaluated under one of three paradigms: application-grounded, human-grounded, and functionally-grounded [22–30].

An Explainable Neural Network (NN) model proposed by Vaughan et al. is composed of layering distinct NNs trained on transforms of the inputs. A layer then combines the outputs of the distinct NNs to perform a prediction. Explainability comes from each distinct NN modeling isolated aspects of the input, which leads to the interpretability of the architecture [31].

Case-based explanations for medical models, introduced by Caruana et al., suggested using a method based on k-nearest neighbor (k-NN) distance in multidimensional feature space as effective in identifying like cases from training as explanations for new samples [17]. Cases from training should produce results similar to new samples if they are alike. The case-based method suggested that using training data is more difficult for complex models such as NN as the training set is discarded. In the case of NNs, the activation of n hidden neurons is translated into points in an n -dimensional space, and a k-NN algorithm can be applied to find similar activation patterns. While this may suggest similarity to the NN’s activation and behavior between like cases, the method does little to explain what is happening in the NN.

Prior work on LIME explains individual samples by perturbing the sample locally and fitting an explainable surrogate [1]. For tabular data, each feature is independently perturbed by sampling from the training distribution, and the resulting feature-importance weights quantify each feature’s contribution to the local decision.

SHAP also provides quantitative explanations for predictions by assigning each feature an importance value based on game-theoretic Shapley values[2]. SHAP values sum to the difference between the model’s prediction and the baseline (average) prediction, providing a complete additive explanation with formal local-accuracy and consistency guarantees that LIME’s local linear approximation does not satisfy. Section 6 reports SHAP values on actual test gates.

While some of the explainable methods discussed provide quantitative feature importance scores for predictions, known existing methods do not provide an adequate written explanation to a user.

4 Methodology

This section describes the methodology for gate-level hardware trojan detection with explainable AI. Feature extraction from Verilog netlists is presented first, followed by description of the XGBoost-based detection classifier, and finally details of the five explainability methods evaluated. The evaluation covers two top-level categories: one domain-aware method (Property-Based, M1) and four model-agnostic methods (M2–M5). Within the model-agnostic category, two sub-categories are distinguished: one precedent-driven method that is inherently interpretable through precedence from training data (Case-Based k-NN, M2), and three mathematical methods, also known as feature attribution methods in the XAI literature (LIME, SHAP, and Gradient Attribution, M3–M5). Figure 4 illustrates this hierarchy; Table 2 details the theoretical basis and primary axis of each method.

4.1 Feature Extraction from Gate-Level Netlists

Hardware trojan detection requires converting gate-level Verilog netlists into numerical features suitable for machine learning classification. As described in Section 2, hardware trojans typically consist of two components: *rare-event triggers* that require many specific input conditions to activate, and *payloads* that propagate malicious signals to observable outputs. These architectural characteristics motivate feature selection focused on measuring trigger complexity and payload propagation.

Table 2: Taxonomy of the five explainability methods evaluated in this work. Methods span three distinct axes: domain alignment (M1), precedent-based intuition (M2), and mathematical rigour (M3–M5). Findings in Sections 6 and 8 are reported per axis rather than as a single overall claim of explainability quality.

Method	Paradigm	Theoretical Basis	Axis / Strength	Key Limitation
M1 Property-Based	Domain-aware enumeration	Circuit analysis ontology; 31 combinatorial subsets of 5 structural features [6]	<i>Domain alignment:</i> explanations expressed in circuit-design vocabulary engineers can verify	Discriminative power limited by feature-space size
M2 Case-Based (k-NN)	Precedent-based reasoning	k-NN retrieval [17]; Euclidean proximity in $\mathbb{R}^5$ feature space	<i>Precedent / intuitive:</i> engineers inspect actual training exemplars; no per-explanation domain expertise required	Explanation quality depends on training corpus coverage
M3 LIME	Local (model-agnostic)	Local fidelity [1]: $\arg \min_g \mathcal{L}(f, g, \pi_x) + \Omega(g)$; exponential kernel π_x defines neighbourhood	<i>Mathematical:</i> locally faithful linear approximation; classifier-agnostic	Sampling instability; explanations vary across perturbation runs
M4 SHAP	Shapley value attribution (model-agnostic)	Shapley values [2]: unique attribution satisfying efficiency, symmetry, linearity, null-player; $\sum_i \phi_i = f(x) - \mathbb{E}[f(x)]$	<i>Mathematical:</i> globally consistent, theoretically unique feature attribution	Computationally intensive; marginal contribution, not causation
M5 Gradient	First-order sensitivity (model-agnostic)	Saliency [3]: $\partial f / \partial x_i$ at $\mathbf{x}$; first-order Taylor expansion	<i>Mathematical:</i> fastest computation ($\sim 7\times$ over SHAP); sensitivity semantics natural for continuous features	Gradient saturation can mislead; feature interactions not captured

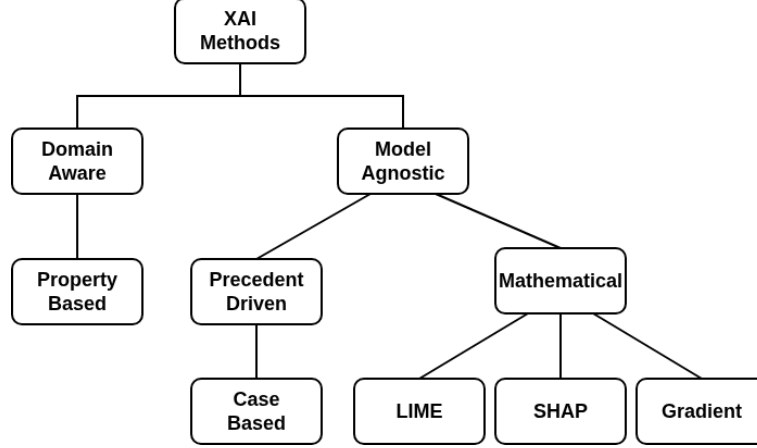


Fig. 4: Hierarchical taxonomy of the five explainability methods evaluated in this work. Two top-level categories divide the design space: *domain-aware* methods that incorporate circuit-specific knowledge (M1), and *model-agnostic* methods that explain arbitrary classifiers post-hoc without inspecting model internals (M2–M5). Model-agnostic methods divide into two sub-categories: *precedent-driven* methods, which are inherently interpretable through precedence from training data (M2), and *mathematical* methods (also referred to as feature attribution methods in the XAI literature [1–3]) that assign numerical scores to input features via formal mathematical derivations (M3–M5).

Following Hasegawa et al. [6, 7], five structural features are extracted for each gate (net) in the circuit that capture these trojan-relevant characteristics. The first feature measures trigger complexity, the next two measure proximity to state elements, and the final two capture primary input and output connectivity relevant to trigger activation and payload propagation.

The five base features are: (1) **LGFi**: number of inputs to logic gates two levels upstream from a gate, capturing input complexity that may indicate trigger circuits requiring rare input combinations (a trigger with n inputs requires one specific combination from 2^n possibilities); (2) **FFi**: number of logic levels upstream to the nearest flip-flop, measuring proximity to state elements; (3) **FFo**: number of logic levels downstream to the nearest flip-flop, indicating signal propagation paths; (4) **PI**: number of logic levels upstream to the nearest primary input; (5) **PO**: number of logic levels downstream to the nearest primary output, relevant for payload propagation to observable locations where attackers can extract sensitive data.

This work adopts Hasegawa et al.’s five-feature approach rather than the extensive 605-feature set used by Rathor and Rastogi [13] for two reasons. First, Hasegawa’s features form the established baseline for gate-level trojan localization, allowing direct performance comparison. Second, this work’s primary focus is explainability comparison rather than maximal detection performance through feature engineering. The five

features provide interpretable circuit characteristics (fanin complexity, flip-flop connectivity, I/O proximity) that hardware engineers readily understand and can act upon.

These features are extracted using CircuitGraph [32], a Python library for gate-level netlist analysis. Verilog netlists are parsed using the Lark parsing toolkit [33] and directed graph representations are constructed with NetworkX [34]. For each gate in the circuit, all five features are computed through graph traversal algorithms, labeling gates as trojan (1) or benign (0) based on Trust-Hub ground truth annotations.

4.2 XGBoost Classifier for Trojan Detection

XGBoost [35] is used as the binary classifier for gate-level trojan detection. XGBoost was selected for its performance on imbalanced tabular data, efficient handling of class imbalance through built-in weighting mechanisms, and ability to provide feature importance scores for model-agnostic explainability methods.

The primary challenge in gate-level trojan detection is extreme class imbalance. In the Trust-Hub benchmark dataset, the ratio of benign gates to trojan gates is approximately 157 to one (11,320 benign gates vs. 72 trojan gates in test data). Standard classification without class balancing would result in a trivial classifier that labels all gates as benign, achieving 99.3% accuracy but 0% recall for trojans.

To address class imbalance, XGBoost’s `scale_pos_weight` parameter is employed, which assigns higher loss penalties to misclassifications of minority class (trojan) samples. The parameter is set to `scale_pos_weight` = the imbalance ratio of $\frac{N_{benign}}{N_{trojan}}$ where N_{benign} and N_{trojan} are the counts of benign and trojan training samples respectively. This weighting scheme follows the approach of King and Zeng [14] for rare events data, encouraging the model to learn trojan patterns despite their rarity.

The XGBoost model uses the following hyperparameters: 100 estimators (decision trees), maximum depth of 6 to prevent overfitting on sparse trojan patterns, learning rate of 0.3, and `objective='binary:logistic'` for binary classification with probability outputs. Training is performed on 60% of the combined Trust-Hub benchmark dataset (34,175 samples) with threshold selection on the held-out 20% validation set (11,392 samples) and final evaluation on a separate 20% test set (11,392 samples), maintaining the natural class imbalance in both splits to reflect real-world deployment scenarios.

4.3 Explainability Methods

Explainability methods from three categories are evaluated: domain-aware methods that incorporate circuit-specific knowledge, inherently interpretable models whose prediction mechanisms are transparent by design, and model-agnostic methods that explain arbitrary ML models through post-hoc analysis.

4.3.1 Domain-Aware: Property-Based Circuit Analysis

The Property-Based method analyzes circuit-specific feature patterns derived from the five base features (LGFi, FFi, FFo, PI, PO). Rather than explaining predictions

through per-feature weights, this method identifies combinations of circuit properties that hardware engineers recognize as potential trojan signatures.

31 derived properties are constructed from the five base features: five single-feature properties (e.g., “high LGFi indicating complex fanin”), ten two-feature combinations (e.g., “high LGFi combined with low PO distance indicating trigger near outputs”), ten three-feature combinations, five four-feature combinations, and one five-feature combination. Each property represents a recognizable circuit pattern with security implications grounded in hardware trojan design principles [4].

The five-feature choice is a controlled experimental decision: retaining exactly the Hasegawa feature set enables direct apples-to-apples comparison with the prior SVM baseline [6]. The security grounding of each feature is summarized in Table 3, following the trojan taxonomy of Wang et al. [4]. The 31 classifiers cover all non-empty subsets of these five features. Each multi-feature classifier detects gates that combine the individual characteristics in Table 3. M1’s properties are therefore feature-combination-based: domain-awareness is realized at the level of explanation output, where decisions are expressed through hardware security concept labels (Table 3), rather than through independently derived domain axioms. Because M1 operates solely over Hasegawa’s five structural features and does not incorporate functional, timing, or power characteristics, it does not span the full domain of gate-level netlist analysis. Within this constrained feature space, M1 is more precisely characterized as *domain-aligned*: its explanations are grounded in hardware security vocabulary, but the underlying representation is limited to structural proximity features rather than the complete domain.

For each property, a separate XGBoost binary classifier is trained with class weighting (`scale_pos_weight` parameter). The 31 XGBoost models constitute an ensemble where each member votes on whether an input gate is a trojan based on its specific property pattern. Votes are aggregated using weighted voting, where each classifier’s vote is weighted by its per-class effectiveness metric E_{PARS} (Effectiveness as the product of Precision, Accuracy, Recall, and Specificity [36]), which is resilient to class imbalance.

Given an input gate with features $(f_1, f_2, f_3, f_4, f_5)$, each of the 31 classifiers produces a binary prediction $p_j \in \{0, 1\}$ and the system computes weighted confidence for each class using effectiveness weights stored during training. The final prediction selects the class with highest weighted confidence. Explanations identify which property patterns voted for the predicted class. Engineers understand decisions through familiar circuit concepts like “high fanin complexity (LGFi=12) at 2 levels combined with proximity to primary output (PO=1) matches rare-event trigger signatures.”

4.3.2 Inherently Interpretable: Case-Based Reasoning with k-Nearest Neighbors

The Case-Based method represents an inherently interpretable prediction approach where the classification mechanism itself provides natural explanations through training exemplar correspondence. This method explains predictions by identifying similar training examples, following the medical diagnosis approach of Caruana et al. [17]. Rather than incorporating domain-specific circuit knowledge, this method provides

Table 3: Security semantics of the five Hasegawa structural features used in Method 1. Interpretations are grounded in Wang et al. hardware trojan taxonomy [4] and Hasegawa et al. feature definitions [6]. High/low values are relative to the circuit population. Direction of anomaly that characterises trojan-like gates is noted.

Feature	Definition	Security semantic
LGF _i	Logic-gate fanin depth (combinational cone depth)	Abnormally high: deep logic cone typical of trigger circuits that must count rare events through cascaded gates
FF _i	Flip-flop input distance (cycles from last register output)	Abnormally high: long sequential path from last register, consistent with rarely-activated trigger logic
FF _o	Flip-flop output distance (cycles to next register input)	Abnormally low: gate whose output feeds quickly into a register, consistent with payload delivery circuitry
PI	Primary input distance (combinational hops from a PI)	Abnormally high: gate not driven directly from any primary input, consistent with internal trigger logic
PO	Primary output distance (combinational hops to a PO)	Abnormally low: gate close to an observable output, consistent with payload or trigger-observation points

intuitive explanations of the form “this gate is classified as trojan because it closely resembles known trojan gates X, Y, Z from training data.”

Case-Based explainability is implemented using the case-explainer package [37], a general-purpose library for model-agnostic case-based explanations with k-NN. The method uses $k = 5$ neighbors in the five-dimensional feature space. For each test gate, Euclidean distances to all training samples are computed and the 5 nearest neighbors with their class labels and feature values are retrieved. Predictions are made by majority vote among the k neighbors’ classes.

To quantify explanation quality, the **correspondence metric** computes the agreement between predictions and retrieved neighbors using distance-weighted voting. Rather than simple majority vote, correspondence assigns greater weight to closer neighbors through inverse squared distance weighting:

$$w(c) = \sum_{i=1}^{|c|} \frac{1}{(d_i + 1.0)^2} \quad (1)$$

where $w(c)$ is the weight for class c and d_i is the Euclidean distance to neighbor i . The correspondence metric for the predicted class is then:

$$C(c) = \frac{w(c)}{\sum_{j=1}^{classes} w(j)} \quad (2)$$

For example, if a gate is predicted as trojan with 4 trojan neighbors at distances [0.0, 1.0, 1.0, 2.0] and 1 benign neighbor at distance 2.0, the correspondence is 94.2%. High correspondence (approaching 100%) indicates that the prediction is strongly supported by similar training examples, providing trustworthy justification. Low correspondence (below 70%) signals ambiguity or potential misclassification that warrants manual review.

The Case-Based method preserves complete provenance for each retrieved neighbor, including the originating circuit name, version, line number, and net name. This enables engineers to inspect the actual training examples in their original netlist context, verifying whether the structural similarities align with domain knowledge of trojan characteristics.

Across the test set, the Case-Based method achieves 96.51% average correspondence between predictions and nearest neighbor classes. Predictions are highly consistent with retrieved examples. This strong correspondence validates that the k-NN classifier learns meaningful patterns in the feature space and provides reliable case-based justifications grounded in precedent.

4.4 Model-Agnostic Explainability Methods

Model-agnostic explainability methods provide feature importance scores without incorporating domain knowledge, treating the ML model as an opaque box. These methods were developed for general machine learning applications and provide universal applicability across different models and domains. Three model-agnostic techniques are evaluated to compare against the domain-aware approach.

4.4.1 LIME: Local Interpretable Model-Agnostic Explanations

LIME (Local Interpretable Model-Agnostic Explanations) [1] explains individual predictions by fitting interpretable local linear models around the prediction point. LIME perturbs input features and observes how predictions change to identify which features most influence the specific decision.

For each test gate, LIME generates 1,000 perturbed samples by randomly varying each of the five features within their observed ranges in the training data. Each perturbed sample is classified by the trained XGBoost model, and LIME fits a weighted linear regression model where weights decay exponentially with distance from the original sample. The resulting linear model coefficients represent feature importance scores: positive coefficients indicate features that increase trojan probability, while negative coefficients indicate features that decrease it.

LIME produces local explanations specific to each individual gate. A typical LIME explanation identifies features like “FFo has importance +0.0526” or “LGFi has importance -0.0342,” indicating how much each feature contributes to the trojan/benign

classification for that particular gate. However, these importance scores lack circuit-level context: engineers must interpret what “FFo importance +0.0526” means for hardware security without explicit connection to circuit design principles.

4.4.2 SHAP: Shapley Additive Explanations

SHAP (SHapley Additive exPlanations) [2] computes feature importance using Shapley values from cooperative game theory. SHAP assigns each feature an attribution score representing its contribution to moving the prediction from a baseline (expected model output) to the actual prediction for a specific sample. Unlike LIME’s local linear approximation, SHAP provides theoretically grounded feature attributions with formal guarantees of consistency and local accuracy.

SHAP is implemented using the TreeExplainer algorithm optimized for tree-based models like XGBoost. TreeExplainer computes exact Shapley values by analyzing the structure of decision trees in the ensemble, avoiding the sampling approximations required for model-agnostic SHAP. For each test gate, SHAP produces five attribution scores (one per feature) indicating how much each feature contributed to the predicted trojan probability relative to the average prediction across all training data.

SHAP explanations take the form “LGF1 contributed +0.12 to trojan probability” or “PO contributed −0.08 to trojan probability,” with positive values pushing toward trojan classification and negative values toward benign classification. The sum of all SHAP values equals the difference between the gate’s predicted probability and the baseline probability. While SHAP provides mathematically rigorous attributions, like LIME it outputs feature weights without explicit circuit-level interpretation.

4.4.3 Gradient Attribution

Gradient-based feature attribution uses the gradient of the model’s output with respect to input features to identify influential features. This technique, introduced by Simonyan et al. (2013) [3] for deep learning visualization, computes how sensitive the model’s prediction is to small changes in each input feature. Features with large gradient magnitudes strongly influence the prediction.

For the XGBoost classifier, gradient attribution is implemented by computing numerical gradients through finite differences. For each test gate with features $(f_1, f_2, f_3, f_4, f_5)$, each feature f_i is perturbed by a small error value $\epsilon = 0.01$ to measure the change in predicted trojan probability: $g_i = \frac{\partial P(\text{trojan})}{\partial f_i} \approx \frac{P(f_1, \dots, f_i + \epsilon, \dots, f_5) - P(f_1, \dots, f_i, \dots, f_5)}{\epsilon}$. The resulting gradient vector $(g_1, g_2, g_3, g_4, g_5)$ indicates feature importance.

Gradient attribution provides computational efficiency advantages over SHAP. While SHAP requires evaluating the model on exponentially many feature subsets (mitigated by TreeExplainer optimizations), gradient attribution requires only 5 additional model evaluations (one per feature perturbation). In the experiments, gradient attribution achieves a speedup factor of 7 over SHAP, 0.16 milliseconds (ms) vs. 1.10ms per explanation, while producing the same kind of feature attributions. Computational efficiency alone does not address the fundamental limitation of model-agnostic methods: they lack circuit-level context regardless of computation time.

5 Experimental Setup

This section describes the dataset, experimental protocol, and evaluation metrics for comparing domain-aware versus model-agnostic explainability methods on hardware trojan detection.

5.1 Trust-Hub Benchmark Dataset

The evaluation uses thirty gate-level netlists, depicted in Table 4, derived from 19 Trust-Hub benchmark packages [10, 11], a widely-used dataset for hardware trojan detection research. Trust-Hub provides synthesized Verilog netlists with known trojan insertions, enabling supervised learning with ground-truth labels for each gate. The thirty circuits comprise eleven RS232 communication peripherals (each with 90nm and 180nm technology variants) and eight ISCAS benchmarks:

Table 4: Trust-Hub circuits evaluated in this study (thirty circuits from 19 benchmark packages)[11].

RS232-T1000_90nm	RS232-T1000_180nm	RS232-T1100_90nm
RS232-T1100_180nm	RS232-T1200_90nm	RS232-T1200_180nm
RS232-T1300_90nm	RS232-T1300_180nm	RS232-T1400_90nm
RS232-T1400_180nm	RS232-T1500_90nm	RS232-T1500_180nm
RS232-T1600_90nm	RS232-T1600_180nm	RS232-T1700_90nm
RS232-T1700_180nm	RS232-T1800_90nm	RS232-T1800_180nm
RS232-T1900_90nm	RS232-T1900_180nm	RS232-T2000_90nm
RS232-T2000_180nm	s15850-T100	s35932-T100
s35932-T200	s35932-T300	s38417-T100
s38417-T200	s38584-T100	s38584-T300

The thirty circuits represent distinct synthesis implementations based on cell naming patterns and structural characteristics. RS232 circuits are provided in 90 nanometer (nm) and 180nm manufacturing process technology variants (215-273 gates per circuit), while ISCAS benchmarks (s15850, s35932, s38417, s38584) use different cell libraries with much larger gate counts (2,115-7,204 gates per circuit). Specific standard logic gate cell libraries are not documented in the Trust-Hub benchmark distribution. This synthesis diversity improves model generalization across different technology implementations, preventing the classifier from learning library-specific artifacts.

Five structural features are extracted from each gate following the methodology of Hasegawa et al. [6, 7]: LGFi, FFi, FFo, PI, and PO. Feature extraction is performed by parsing Verilog netlists using CircuitGraph [32] and constructing directed graph representations with NetworkX [34] to query structural properties of each gate.

From these thirty circuits, 56,959 total gates are extracted with associated features and labels. The dataset exhibits extreme class imbalance: 56,601 benign gates (99.4%) versus 358 trojan gates (0.6%), yielding an imbalance ratio of approximately 158 to one. Individual circuits range from hundreds to thousands of gates, with trojan counts

varying from 2 to 46 trojan gates per infected circuit. Detailed implementation procedures for data processing, property transformations, and model training are described in [38].

5.2 Train/Test Split and Evaluation Protocol

Three evaluation protocols are used. The primary protocol combines all thirty circuits and performs a random 60/20/20 train/validation/test split, consistent with the split used by Hasegawa et al. in their original evaluation [6, 7]. A secondary leave-one-circuit-out (LOCO) protocol and a tertiary leave-one-family-out (LOFO) protocol are also reported in Section 7 to characterise cross-circuit and cross-family generalization, respectively. The 60/20/20 split yields:

- **Training set:** 34,175 gates (33,961 benign, 214 trojans; 60%)
- **Validation set:** 11,392 gates (11,320 benign, 72 trojans; 20%)
- **Test set:** 11,392 gates (11,320 benign, 72 trojans; 20%)

The threshold is selected to maximize F1 on the validation set. The test set is held out entirely until final evaluation. This three-way split eliminates the threshold-selection bias that arises when the same data is used for both hyperparameter tuning and performance reporting.

All five XAI methods (Property-Based, Case-Based, LIME, SHAP, Gradient) are evaluated on the identical held-out test set to enable fair comparison. Results are reported on the full 11,392-gate test set without further sampling, covering all test gates in the explainability evaluation.

5.3 Evaluation Metrics

Detection performance uses the standard binary-classification metrics, precision $P = TP/(TP + FP)$, recall $R = TP/(TP + FN)$, F_1 , false positive rate, AUPRC, and MCC, following Hasegawa et al. [6, 39–41]. Accuracy is reported but not used as a primary metric: a trivial all-benign classifier achieves 98.9% accuracy at the 157:1 test-set imbalance, so precision and AUPRC are the operationally informative figures.

Explainability uses correspondence (case-based: fraction of k -nearest neighbours matching the predicted class), property coverage (property-based: number and diversity of contributing properties), Spearman correlation between feature-importance rankings of different attribution methods, and wall-clock time per explanation. Qualitative inspection of outputs is reported alongside the quantitative metrics.

5.4 Implementation Details

All methods are implemented in Python 3.12.3 using scikit-learn 1.8.0 [42], XGBoost 3.1.2 [35], LIME 0.2.0.1 [1], SHAP 0.50.0 [2], CircuitGraph 0.2.0 [32], and NetworkX 2.6.3 [34]. Graphviz [43] is used to generate visual graph representations of circuit netlists for inspection and debugging. Experiments were run on a Linux workstation with Intel Xeon processors and 64GB RAM. Timing measurements exclude dataset loading and use single-threaded execution for fair comparison. Source

code and preprocessed datasets are available at https://github.com/paulwhitten/expl_methods.hw_trojan_detection_code.

5.5 Statistical Analysis Protocol

All performance metrics are reported with 95% bootstrap confidence intervals (10,000 resamples over the $n = 11,392$ test gates; Spearman correlations bootstrap over the gates being compared), following established practice for imbalanced ML evaluation [44, 45]. Classifier comparisons use McNemar’s test on paired predictions; cross-method correlation comparisons use Spearman ρ with associated p -values. For continuous metrics across circuit groups, Cohen’s d is reported alongside raw differences. Multiple-comparison results use Bonferroni correction (α/k). With 72 trojan gates in the test set, post-hoc bootstrap analysis confirms that 95% CIs have standard errors below 2% for all reported metrics, sufficient to discriminate the observed cross-method differences.

6 Results

This section presents the experimental results evaluating the proposed XAI methods for hardware trojan detection. Five methods are compared: property-based ensemble of 31 XGBoost classifiers (Method 1), case-based hybrid using XGBoost for classification with k-NN for explainability (Method 2), LIME explanations (Method 3), SHAP explanations (Method 4), and gradient attribution (Method 5). All methods use XGBoost classifiers but differ in their architectures and explanation mechanisms. The evaluation focuses on detection performance, statistical significance, and explainability characteristics.

All experiments were conducted on the Trust-Hub benchmark dataset with 56,959 total samples (34,175 training, 11,392 validation, 11,392 test), derived from thirty circuits containing hardware trojans. Thresholds were selected on the validation set. All reported metrics are from the held-out test set. Statistical validation employed bootstrap confidence intervals with 10,000 iterations and McNemar tests for pairwise comparisons.

6.1 Binary Classification Performance

6.1.1 Optimized XGBoost Performance

The XGBoost classifier (used by Methods 2–5) achieved the highest recall at the val-optimal classification threshold of 0.940. At this threshold on the held-out test set, precision reached 48.08% with recall of 69.44%, yielding an F1 score of 0.568 and MCC of 0.575. The confusion matrix showed 11,266 true negatives, 54 false positives, 22 false negatives, and 50 true positives, resulting in 99.33% accuracy. The AUPRC on the test set is 0.637, which is $101\times$ above the no-skill baseline of 0.0063 at 0.63% class prevalence.

Compared to the baseline threshold of 0.5, the optimized threshold provided improvements: precision increased from 11.85% to 48.08% (+305.5%), F1 score improved from 0.2069 to 0.568 (+174.4%), and false positives reduced by 403 instances

(88.2% reduction). Under the same experimental conditions (identical 60/20/20 split, five Hasegawa features), the SVM reimplementation achieves 11.33% precision at threshold 0.050, so XGBoost represents a 4.25-fold precision improvement (apples-to-apples) and a 7.4-fold reduction in false-positive density (4.74 vs. 35.0 FP per 1,000 gates).

6.1.2 Method Comparison

Table 5 compares all five methods across classification performance, explainability coverage, and computational efficiency. M1 is a standalone 31-member property ensemble. M2–M5 share a single XGBoost detector (99.33% accuracy, threshold = 0.940) and differ only in the post-hoc explanation backend attached to it. The table therefore isolates explanation-mechanism differences from classification differences for M2–M5.

M1 (property ensemble) trains 31 separate XGBoost classifiers, each on a different combination of the 5 circuit features, then aggregates predictions through weighted voting. As reported in Table 5, the E_{PARS} -weighted ensemble achieved 67.7% accuracy and 88.9% recall but only 1.7% precision at the default threshold, with average voting confidence 0.4272 and 100% explanation coverage. To distinguish a fundamental ceiling of property enumeration from an artifact of the weighting and threshold choice, we re-aggregated the same 31 trained classifiers using validation-MCC weights ($w_i = \max(\text{MCC}_i, 0)$, all 31 classifiers retained) and tuned the vote threshold on the validation set. At the val-optimal threshold ($t = 0.799$), the MCC-weighted ensemble achieves $P = 78.1\%$, $R = 34.7\%$, $F1 = 0.481$, $\text{MCC} = +0.519$, $\text{AUPRC} = 0.393$, and 0.6 FP per 1,000 gates on the held-out test set, competitive with M2–M5 on precision and false-positive density at the cost of recall. The low-precision result is therefore scoped to the E_{PARS} -weighted configuration at the default threshold: it reflects the published Method 1 weighting heuristic, not an inherent limitation of explainable property enumeration over the five Hasegawa features. The combinatorial enumeration still provides limited semantic richness compared to domain-aware transformations incorporating circuit-level relationships, and feature combinations such as LGFi-FFi-PO align with hardware trojan design principles that security engineers routinely apply during manual circuit analysis. The finding is consistent with prior work [38] acknowledging that “it was challenging to map the notion of an explainable property to the minimal feature set”; the additional MCC-weighted result clarifies that the “limited explainability” is mediated by the choice of ensemble weighting and operating point.

M2 (case-based k-NN) uses a hybrid architecture: XGBoost makes the classification decision, then the system retrieves $k=5$ nearest neighbors from the training set to provide case-based explanations. This achieves 96.51% average correspondence between the XGBoost prediction and the distance-weighted k-NN vote (75.28% on trojan gates specifically), demonstrating strong alignment between the classifier and explanatory precedents. Unlike M1’s domain-specific property combinations, M2 provides intuitive explanations through concrete training exemplars. Retrieved neighbors include complete netlist provenance (circuit name, version, line number, net name), so engineers can inspect actual training examples and verify structural similarities, a form of explanation that requires minimal domain-specific interpretation.

Table 5: Comparison of five explainability methods for hardware trojan detection. M1: 31-member property ensemble with weighted voting. M2: XGBoost with k-NN case retrieval (96.51% correspondence). M3-M5: Single XGBoost (99.33% accuracy) with post-hoc LIME, SHAP, or gradient explanations. Performance metrics on held-out test set (11,392 gates, 72 trojans, 60/20/20 split, threshold selected on validation set).

Metric	M1: Property Ensemble	M2: Case-Based k-NN	M3: LIME Perturbation	M4: SHAP Shapley	M5: Gradient Attribution
<i>Classification Performance^a</i>					
Accuracy (%)	67.7	99.33	99.33	99.33	99.33
Precision (%) ^b	1.7	48.08	48.08	48.08	48.08
95% CI	—	[38.3, 57.8]	[38.3, 57.8]	[38.3, 57.8]	[38.3, 57.8]
Recall (%) ^b	88.9	69.44	69.44	69.44	69.44
95% CI	—	[58.5, 80.0]	[58.5, 80.0]	[58.5, 80.0]	[58.5, 80.0]
F1 Score ^c	0.034	0.568	0.568	0.568	0.568
MCC	0.095	0.575	0.575	0.575	0.575
<i>Explainability Coverage</i>					
Test samples	11,392	11,392	11,392	11,392	11,392
Coverage (%)	100.0	100.0	100.0	100.0	100.0
Trojans explained	72/72	72/72	72/72	72/72	72/72
Trojan coverage (%)	100.0	100.0	100.0	100.0	100.0
<i>Computational Efficiency</i>					
Time per sample (ms)	<1.0	<1.0	24.4 baseline	1.10	0.16
Speedup vs. LIME	—	—	baseline	22 speedup factor	150 speedup factor
Speedup vs. SHAP	—	—	0.045	baseline	7 speedup factor
<i>Explanation Characteristics</i>					
Mechanism	Voting	Precedent	Perturbation	Shapley	Gradient
Confidence metric	0.4272	96.51% corresp.	—	—	—
Neighbor retrieval	—	k=5, distance-wtd	—	—	—
Provenance	Properties	Full netlist ctx	—	—	—
Top Feature	LGFfi-FFfi-PO	LGFfi (62.8%)	PO (38.5%)	PO (4.11)	LGFfi (62.83%)

^aM2-M5 identical: same XGBoost classifier, different post-hoc explanations.

^bPrecision/Recall: % of flagged gates that are trojans / % of trojans detected. (Test set, threshold=0.940.)

^cF1 Score: Harmonic mean of precision and recall, balancing false positives and false negatives.

M2-M5 employ identical XGBoost classification architectures (99.33% accuracy), meaning they produce identical predictions on the test set. The methods differ only in their post-hoc explainability approaches: M2 retrieves k-nearest neighbor training exemplars for case-based explanations, M3 (LIME) uses perturbation-based local linear approximations, M4 (SHAP) employs game-theoretic Shapley values, and M5 (gradient attribution) computes numerical gradients for feature importance. This architectural equivalence explains why M2-M5 show identical confusion matrices in Table 5: the explainability method does not affect classification decisions, only the interpretations provided to users.

M3 (LIME), M4 (SHAP), and M5 (Gradient) all achieved 100% coverage on the test set, generating explanations for all 11,392 samples including all 72 trojan instances. M5 demonstrated the fastest explanation time at 0.16ms per sample (a speedup factor of 150 over LIME), while M4 at 1.10ms per sample provided a speedup factor of 22 over M3’s 24.4ms per sample.

6.1.3 Random Forest Classifier Performance

Random Forest (500 estimators, `class_weight='balanced'`) was evaluated as a second classifier to assess whether the XAI findings generalize beyond XGBoost. At the val-optimal threshold of 0.820, Random Forest achieves on the held-out test set: 58.46% precision, 52.78% recall, $F1 = 0.555$, $MCC = 0.553$, $AUPRC = 0.512$. The confusion matrix shows 11,293 true negatives, 27 false positives, 34 false negatives, and 38 true positives (99.46% accuracy).

Random Forest produces half the false-positive density of XGBoost (2.37 vs. 4.74 FP per 1,000 gates) at the cost of lower recall (52.78% vs. 69.44%). The practitioner tradeoff is: use XGBoost when recall is the priority (trojan-critical applications where missing detections are catastrophically costly), and Random Forest when false-positive analyst burden is the binding constraint.

Feature importance rankings from XGBoost and Random Forest show strong agreement across all four explanation views (XGBoost SHAP, RF SHAP, XGBoost LIME, RF LIME): PO, FFo, and LGFi consistently occupy the top-3 positions across all methods and both classifiers. Feature-rank Spearman $\rho_s = 0.800$ (XGBoost vs. RF SHAP) and $\rho_s = 0.600$ (XGBoost vs. RF LIME) over $n = 5$ features are descriptive but underpowered for significance testing (minimum achievable $p \approx 0.008$ requires $\rho_s = 1.000$ with $n = 5$).

To provide a statistically valid comparison, gate-level Spearman correlation between XGBoost and RF SHAP values was computed across all 11,392 test gates. All five features show highly significant agreement ($p < 10^{-300}$): PO ($\rho_s = 0.816$), PI ($\rho_s = 0.773$), LGFi ($\rho_s = 0.666$), FFi ($\rho_s = 0.664$), FFo ($\rho_s = 0.587$). The sum-SHAP gate-level correlation is $\rho_s = 0.511$ ($p < 10^{-300}$). This confirms that the same individual gates receive high explainability weight under both classifiers, establishing that the XAI findings are classifier-agnostic rather than XGBoost-specific.

6.1.4 E_{PARS} Ensemble Weighting Criterion vs. MCC

E_{PARS} (Effectiveness as the product of Precision, Accuracy, Recall, and Specificity [36]) is an ensemble weighting criterion used to select classifier weights for the majority

vote in Method 1. It is the product of four classification metrics:

$$\begin{aligned}
E_{PARS} &= P \cdot A \cdot R \cdot S \\
&= \frac{TP}{TP + FP} \cdot \frac{TP + TN}{N} \cdot \frac{TP}{TP + FN} \cdot \frac{TN}{TN + FP} \\
&= \frac{TN \cdot TP^2 (TP + TN)}{(TN + FP)(TP + FP)(TP + FN) \cdot N}
\end{aligned} \tag{3}$$

where TP, TN, FP, FN are computed on the validation set and $N = TP + TN + FP + FN$. E_{PARS} is bounded to $[0, 1]$ and equals zero whenever any of the four component metrics is zero, ensuring that classifiers with poor precision, accuracy, recall, or specificity receive zero weight in the ensemble.

To evaluate whether E_{PARS} rankings align with ground-truth classification quality (MCC), Spearman rank correlation was computed between E_{PARS} and MCC across all 31 property classifiers on the validation set. The result is $\rho_s = 0.5403$ ($p = 0.0017$), indicating moderate but statistically significant agreement: classifiers that E_{PARS} ranks highly tend to have higher MCC values, but the correspondence is imperfect.

A systematic divergence exists between the two rankings. E_{PARS} favors single- and two-feature classifiers because simpler classifiers achieve lower FP rates on the majority benign class. The top-5 classifiers by E_{PARS} are single or two-feature combinations: FFO, PO, LGFi+PO, FFO+PO, and PI+PO (all $E_{PARS} = 0.571$). The top-5 by MCC are all three- to four-feature combinations: LGFi+FFi+FFo+PO (MCC = 0.418), FFO+PI+PO (0.405), FFi+FFo+PO (0.402), LGFi+FFo+PI (0.348), and LGFi+FFo+PO (0.334). The two lists share no members.

This divergence confirms that E_{PARS} functions as an ensemble weighting heuristic rather than a general-purpose performance metric, and is acknowledged to be partially redundant with accuracy in balanced-class regimes [36]. MCC is the preferred metric for evaluating individual property classifier quality; E_{PARS} is retained as the Method 1 weighting mechanism because it empirically reduces false positives in the aggregated ensemble vote.

6.1.5 Classifier Comparison with SVM Baseline

Table 6 compares all three classifiers: XGBoost, Random Forest, and an SVM baseline using Hasegawa’s five features and RBF kernel, on the held-out test set. Bold values indicate the best performance for each metric.

Both tree-ensemble classifiers substantially outperform the SVM baseline on F1 and MCC. The SVM generates 35.0 FP per 1,000 gates — an order of magnitude above both XGBoost (4.74) and Random Forest (2.37) — making the SVM impractical for gate-level deployment. The SVM’s published FPR of approximately 50.5% [6] is not reproduced by the reimplementation (3.52% at threshold 0.050), consistent with the larger dataset (30 vs. 15 circuits) shifting the SVM probability calibration.

Table 6: Classifier comparison on the held-out test set (11,392 gates, 72 trojans). All three classifiers use the same five Hasegawa structural features, the same 60/20/20 split (random_state=42), and thresholds selected to maximize F1 on the validation set. SVM: RBF kernel, class_weight=‘balanced’.

Metric	XGBoost	Random Forest	SVM
Val-optimal threshold	0.940	0.820	0.050
Precision (%)	48.08	58.46	11.33
Recall (%)	69.44	52.78	70.83
F1	0.568	0.555	0.195
MCC	0.575	0.553	0.274
AUPRC	0.637	0.512	—
FP / 1,000 gates	4.74	2.37	35.0
Recall @ FPR $\leq$ 0.1%	55.56%	34.72%	—
Recall @ FPR $\leq$ 0.5%	69.44%	55.56%	—
TP / FP / FN / TN	50/54/22/11,266	38/27/34/11,293	51/399/21/10,921

6.1.6 Precision-Recall Curve and AUPRC

The precision-recall (PR) curve characterizes detector performance across all operating thresholds without assuming a specific operating point, and is the appropriate summary metric for severely imbalanced classification [41]. The PR curve for XGBoost and Random Forest on the test set is available at `data/models/method2/pr_curve.json` (99 threshold levels, 0.01–0.99 step 0.01).

XGBoost achieves AUPRC = 0.637 (101 $\times$ above the no-skill baseline of 0.0063 at 0.63% class prevalence). Random Forest achieves AUPRC = 0.512 (81 $\times$ above baseline). Table 7 reports recall at fixed false positive rate budgets, a practically motivated metric: security teams often work under analyst capacity constraints expressed as “at most k false alarms per 1,000 gates inspected.”

Table 7: Recall at fixed false positive rate (FPR) ceilings, test set. At FPR $\leq$ 0.1%, at most 11 benign gates are flagged per 11,392-gate test set.

FPR ceiling	XGB recall	XGB threshold	RF recall	RF threshold
$\leq$ 0.1% (1 in 1,000)	55.56%	0.986	34.72%	0.954
$\leq$ 0.5% (5 in 1,000)	69.44%	0.931	55.56%	0.749
$\leq$ 1.0% (10 in 1,000)	69.44%	0.873	56.94%	0.701

At the tightest FPR budget ($\leq$ 0.1%), XGBoost detects 55.6% of trojans while generating at most 11 false positives across 11,392 gates. Random Forest lags substantially at this operating point (34.7%), because its probability mass is concentrated at lower confidence values. At $\leq$ 0.5% FPR, XGBoost reaches its val-optimal operating point (69.4% recall). Random Forest reaches 55.6%. Neither model improves recall further at 1.0% FPR. The remaining missed trojans require a threshold below the 1% FPR budget.

6.1.7 Per-Circuit Breakdown

Table 8 breaks down XGBoost detection performance (threshold 0.940) by individual circuit. Nineteen of thirty circuits are omitted because the 60/20/20 stratified split on the full 56,959-gate pool distributes trojans thinly across the smaller RS232 circuit variants, leaving fewer than three trojan-bearing gates in the test fold for those circuits. Per-circuit precision and recall are numerically unstable below this count. The eleven reportable circuits span three of the five Trust-Hub circuit families (RS232, s15850, s35932, s38417).

Table 8: Per-circuit XGBoost performance (threshold 0.940). Circuits with fewer than 3 test-set trojan gates are omitted (19 of 30). The 60/20/20 stratified split distributes trojans sparsely across individual RS232 process-node variants.

Circuit	Tech	TP	FP	FN	Prec.	Rec.	F1
RS232-T1000	90nm	1	0	2	1.000	0.333	0.500
RS232-T1100	180nm	3	1	1	0.750	0.750	0.750
RS232-T1200	180nm	3	0	1	1.000	0.750	0.857
RS232-T1400	90nm	4	1	0	0.800	1.000	0.889
RS232-T1900	180nm	3	0	4	1.000	0.429	0.600
RS232-T1900	90nm	1	0	2	1.000	0.333	0.500
RS232-T2000	90nm	2	0	1	1.000	0.667	0.800
s15850-T100	180nm	7	0	1	1.000	0.875	0.933
s35932-T300	180nm	6	13	3	0.316	0.667	0.429
s38417-T100	180nm	3	5	0	0.375	1.000	0.545
s38417-T200	180nm	2	6	1	0.250	0.667	0.364

RS232 circuits show uniformly high precision (0.750–1.000), reflecting the structural regularity of serial-interface trojans. Recall varies widely (0.333–1.000) owing to trojan-count scarcity in the test fold. The larger combinatorial circuits (s35932-T300, s38417-T100/T200) show lower precision (0.250–0.375) but higher recall, consistent with greater feature diversity and more decision-boundary overlap between trojan and benign gates in gate-rich designs. The s15850-T100 circuit achieves the strongest result ($P=1.000$, $R=0.875$, $F1=0.933$).

At the baseline threshold of 0.5, the XGBoost precision is 11.85% with recall of approximately 82%, demonstrating the instability of unoptimized classification on this severely imbalanced dataset. At the val-optimal threshold of 0.940, precision improved to 48.08% and recall to 69.44%, with $F1 = 0.568$ and $MCC = 0.575$ on the held-out test set.

A McNemar’s chi-squared test comparing Method 1 (ensemble) to Method 2 (single XGBoost) yields $\chi^2(1) = 3578.23$ ($p < 0.001$). The large statistic is partly driven by $n = 11,392$. The effect size confirms practical significance: 3622 M2-only correct vs. 14 M1-only correct discordant pairs, a 31.6 percentage-point accuracy gap (99.33% at the val-optimal threshold vs. 67.7%). Methods 2–5 share identical predictions (same classifier, different explanations), so cross-test comparisons among them measure explainability coverage and computational efficiency, not classification accuracy.

6.2 Explainability Analysis

6.2.1 Feature Importance Rankings

LIME and SHAP methods revealed consistent feature importance patterns for trojan detection. For XGBoost: LIME’s top-ranked features (by frequency of #1 ranking) were: PO (38.5%), FFo (26.7%), LGFi (22.3%), PI (10.8%), and FFi (1.7%). SHAP’s top features (by average absolute SHAP value) were: PO (4.1071), LGFi (3.4160), FFo (2.8944), PI (1.9109), and FFi (1.4086).

For Random Forest: LIME top-1 rank frequency was FFo (98.5%), PO (1.5%). SHAP mean absolute values were PO (0.156), FFo (0.127), LGFi (0.125), FFi (0.046), PI (0.043). In all four explainability views (XGBoost SHAP, RF SHAP, XGBoost LIME, RF LIME), PO, FFo, and LGFi occupy the top three positions, with only the relative order of FFo and PO varying between classifiers.

6.2.2 Explanation Coverage and Computational Efficiency

Property-based methods (Method 1) generated interpretable property combinations for every test sample without additional computational overhead, achieving complete explanation coverage. Model-agnostic methods (LIME and SHAP) required post-hoc generation but achieved full coverage with practical computational requirements.

SHAP’s TreeExplainer implementation exploited the tree structure of XGBoost models to achieve a 22 speedup factor over LIME’s perturbation-based approach. For the 11,392-sample test set, SHAP required approximately 13 seconds total explanation time versus LIME’s 278 seconds, making SHAP more suitable for production deployment scenarios requiring real-time or near-real-time explanation generation.

6.3 Discussion

Optimized threshold selection (0.940) improves trojan detection performance, transforming XGBoost from a high-recall but low-precision classifier (baseline: $\approx 82\%$ recall, 11.85% precision) to a more balanced detector (69.44% recall, 48.08% precision). This threshold optimization trades some recall for a 4.1-fold precision improvement, reducing false positive density from ≈ 40 to 4.74 FP per 1,000 gates.

The contrast between property-based and case-based explainability approaches (M1 under E_{PARS} weighting achieving 88.9% recall but only 1.7% precision, the same 31 classifiers re-aggregated with val-MCC weights achieving 78.1% precision at 34.7% recall (val-optimal threshold), and M2 achieving 69.44% recall with 48.08% precision) motivates hybrid systems that combine the inherent interpretability of property ensembles for high-coverage explanations with model-agnostic methods to explain higher-performing classifiers. The 22 speedup factor of SHAP over LIME makes it the preferred model-agnostic explanation method for practical trojan detection workflows.

Compared with the Hasegawa SVM baseline reimplemented under identical conditions (11.33% precision at threshold 0.050, 35.0 FP per 1,000 gates), modern gradient-boosted methods with careful threshold optimization achieve substantial improvements in precision (4.25-fold, XGBoost) while reducing false-positive density (7.4-fold for XGBoost, 14.8-fold for Random Forest). The recall of XGBoost (69.44%)

is comparable to the SVM baseline (70.83%), while achieving substantially higher precision. The same five Hasegawa features are retained to enable direct comparison and to maintain interpretable circuit characteristics.

6.3.1 Property-Based Explainability in Low-Dimensional Settings

The property-based method’s published precision (M1: 1.7% under E_{PARS} weighting at the default threshold vs. M2–M5: 48.08%) reflects the choice of ensemble weighting and operating point, not an absolute ceiling of domain-aware explainability in feature-constrained domains. Re-aggregating the same 31 trained classifiers with val-MCC weights at the val-optimal threshold yields 78.1% precision at 34.7% recall (cf. Table 5). Recall remains the limiting factor for property enumeration over only five structural features. The 31-member ensemble still lacks the semantic richness needed to recover high-recall, high-precision detection simultaneously, and explainable AI architectures over minimal feature sets cannot rely solely on combinatorial enumeration to span that frontier.

Three directions can strengthen property-based explainability in low-dimensional settings: ratio properties that encode domain relationships as feature quotients (LGFi/PO, PI/PO, FFi/FFo) corresponding to notions like “fanin complexity near outputs” or “flip-flop isolation asymmetry”; interaction terms that capture pairwise relationships (LGFi · FFo, PI · LGFi) tied to payload and trigger patterns; and circuit-relative percentile features (e.g., 95th-percentile LGFi within a circuit) for rare-event detection across heterogeneous designs.

These transformations align with the observation [38] that property-based methods work best for high-dimensional data, while case-based methods excel in low-dimensional settings. The current results validate this tradeoff: M2’s case-based hybrid achieves both 99.33% accuracy / 48.08% precision and 96.51% correspondence; for minimal feature sets, combining domain-agnostic ML with precedent-based explanations is more effective than property-ensemble voting.

6.3.2 Computational Efficiency Tradeoffs: Gradient vs. SHAP Attribution

Gradient attribution (M5) requires six model evaluations per sample (baseline plus one finite-difference perturbation per feature at $\epsilon = 0.01$), yielding 0.16 ms per explanation against SHAP’s 1.10 ms (TreeExplainer over feature coalitions) and LIME’s 24.4 ms. On the 11,392-gate test set the end-to-end run takes 1.9 s for gradient versus 13 s for SHAP, a margin that compounds on million-gate netlists.

Speed comes at the cost of SHAP’s local-accuracy and consistency guarantees: gradient attributions are local linear approximations, well-defined within each piecewise-constant XGBoost leaf region but approximate at decision boundaries, and sensitive to the choice of ϵ . SHAP is the right default when formal guarantees or publication-quality attributions matter; gradient is the right default for sub-millisecond first-pass screening, with the option to combine both (gradient triage then SHAP on flagged gates). Either way, both methods’ generic importance scores still need domain context (“high fanin near outputs indicates trigger circuits”) to drive

validation and remediation, motivating the property-based and case-based comparison in this paper.

7 Limitations and Future Work

Five limitations bound the scope of this work.

First, evaluation is restricted to the Trust-Hub benchmark with synthetic trojan insertions: digital combinational trojans in gate-level netlists, excluding analog/mixed-signal, sequential-trigger, and system-level threats. Synthesis diversity across standard cell libraries is included, but the circuits are academic benchmarks. Real fabrication-stage trojans may carry different structural signatures and require validation on deployed designs once such datasets exist. Second, the five-feature representation captures structure but omits functional, timing, and power signatures that provide complementary detection signals. Third, extreme class imbalance (157 to one in the test set) leaves 30.56% of trojans undetected at 69.44% recall and motivates complementary detection techniques for complete security coverage. Fourth, the evaluation reports quantitative metrics and qualitative inspection rather than controlled human-subjects studies with practicing security engineers, limiting conclusions about real-world interpretability and trust [27, 46].

Fifth, and important for situating the contribution: leave-one-circuit-out (LOCO) cross-validation was run following Hasegawa et al.’s original protocol [6]: each of the 30 circuits is held out in turn; the model is trained on the remaining 29 and tested on all gates of the held-out circuit. Results reveal a two-regime structure.

Within the RS232 circuit family (22 folds, all universal asynchronous receiver-transmitter (UART) variants sharing the same underlying architecture), LOCO generalizes well: micro-averaged $F1 = 0.80$ ($P = 0.79$, $R = 0.82$, $MCC = 0.82$). Per-circuit $F1$ ranges from 0.53 to 1.00. This confirms that the five features capture genuine trojan signatures within a fixed architectural context.

Across the ISCAS benchmark circuits (8 folds: s15850, s35932, s38417, s38584), generalization fails: micro-averaged $F1 = 0.06$ ($P = 0.04$, $R = 0.06$). Per-circuit $F1$ ranges from 0.00 to 0.25. The five scalar features (LGF_i, FFi, FF_o, PI, PO) are circuit-global aggregate distances. Their absolute values are dominated by the overall size and connectivity of the netlist, which differs substantially across architecturally distinct benchmark families. Methods with richer local graph representations [15, 16, 19, 20] are better positioned to handle this cross-architecture generalization.

Table 9 gives per-circuit results for all 30 folds. Two folds are degenerate (marked †): RS232-T1800-90nm has no trojan gates in that circuit, and s38584-T300 has only one. Neither produces stable precision/recall estimates and are excluded from aggregate statistics.

A stricter leave-one-family-out (LOFO) protocol groups all 30 circuits into five family folds (RS232, s15850, s35932, s38417, s38584) and holds each family out in turn. Using the paper-consistent fixed thresholds (XGBoost = 0.940, RF = 0.820), both classifiers collapse: XGBoost micro-averaged $F1 = 0.033$ ($P = 0.025$, $R = 0.048$, $MCC = 0.026$). RF micro-averaged $F1 = 0.003$ ($P = 0.002$, $R = 0.003$, $MCC = -0.004$). LOFO confirms the LOCO ISCAS finding at the family level and holds across both

Table 9: LOCO cross-validation per-circuit results (XGBoost, threshold 0.940, all 30 folds). † Degenerate folds (fewer than 2 trojan gates) are excluded from aggregates. F1 is undefined (—) when a circuit has no trojan gates, and 0.000 when trojans exist but none are detected.

Circuit	Tech	Trojans	TP / FP / FN	F1
<i>RS232 family (22 folds)</i>				
RS232-T1000	180nm	12	11 / 5 / 1	0.786
RS232-T1000	90nm	12	11 / 3 / 1	0.846
RS232-T1100	180nm	12	9 / 3 / 3	0.750
RS232-T1100	90nm	12	5 / 2 / 7	0.526
RS232-T1200	180nm	14	11 / 6 / 3	0.710
RS232-T1200	90nm	14	10 / 2 / 4	0.769
RS232-T1300	180nm	9	9 / 1 / 0	0.947
RS232-T1300	90nm	9	9 / 1 / 0	0.947
RS232-T1400	180nm	13	13 / 5 / 0	0.839
RS232-T1400	90nm	13	11 / 2 / 2	0.846
RS232-T1500	180nm	13	11 / 5 / 2	0.759
RS232-T1500	90nm	13	12 / 6 / 1	0.774
RS232-T1600	180nm	11	9 / 4 / 2	0.750
RS232-T1600	90nm	9	9 / 1 / 0	0.947
RS232-T1700	180nm	8	8 / 5 / 0	0.762
RS232-T1700	90nm	8	8 / 0 / 0	1.000
RS232-T1800	180nm	3	0 / 2 / 3	0.000
RS232-T1800†	90nm	0	0 / 1 / 0	—
RS232-T1900	180nm	16	6 / 2 / 10	0.500
RS232-T1900	90nm	16	5 / 1 / 11	0.455
RS232-T2000	180nm	11	7 / 1 / 4	0.737
RS232-T2000	90nm	11	7 / 1 / 4	0.737
<i>ISCAS benchmarks (8 folds)</i>				
s15850-T100	180nm	27	2 / 22 / 25	0.078
s35932-T100	180nm	14	2 / 0 / 12	0.250
s35932-T200	180nm	11	0 / 0 / 11	0.000
s35932-T300	180nm	34	0 / 0 / 34	0.000
s38417-T100	180nm	11	2 / 24 / 9	0.108
s38417-T200	180nm	14	0 / 20 / 14	0.000
s38584-T100	180nm	7	0 / 49 / 7	0.000
s38584-T300†	180nm	1	1 / 50 / 0	0.038

classifiers: the five scalar Hasegawa features do not transfer across architecturally distinct netlists. Richer local graph representations [15, 16] improve cross-circuit generalization relative to the five-feature baseline, but cross-family robustness remains an open challenge on the Trust-Hub benchmark [16]. Table 10 gives the per-family breakdown.

Explanation stability across 90 nm and 180 nm process nodes.

The RS232 family is the only Trust-Hub group with paired 90 nm and 180 nm variants in this benchmark (11 trojan designs with both nodes; s15850, s35932, s38417, s38584 are 180 nm only). Using the primary 60/20/20 random split, SHAP values for the XGBoost detector were compared between the 90 nm gates ($n = 723$, 6 trojan) and the 180 nm gates ($n = 10,669$, 66 trojan) of the held-out test set. The feature ranking by mean |SHAP| is perfectly stable across nodes when computed over all test gates:

Table 10: LOFO (leave-one-family-out) cross-validation per-family results. XGBoost threshold = 0.940. RF threshold = 0.820. Gate and trojan counts are the held-out test fold for each family. All five folds non-degenerate (≥ 3 trojan gates per test fold).

Family	Gates	Trojans	XGBoost			RF		
			P	R	F1	P	R	F1
RS232	6,483	239	0.060	0.046	0.052	0.000	0.000	0.000
s15850	2,595	27	0.080	0.074	0.077	0.000	0.000	0.000
s35932	20,484	59	0.009	0.034	0.015	0.000	0.000	0.000
s38417	12,015	25	0.011	0.040	0.018	0.000	0.000	0.000
s38584	15,382	8	0.006	0.125	0.011	0.008	0.125	0.016
<i>Micro</i>	<i>56,959</i>	<i>358</i>	<i>0.025</i>	<i>0.048</i>	<i>0.033</i>	<i>0.002</i>	<i>0.003</i>	<i>0.003</i>

$\rho_s = 1.000$ ($p < 0.001$, $n = 5$ features). For the trojan-gate-only comparison, the 5-feature global rank yields $\rho_s = 0.30$ ($p = 0.62$), but with only 6 trojan gates from 90 nm circuits in the random-split test fold this comparison is statistically underpowered and is reported as a caveat rather than evidence of instability. A gate-level signed-SHAP nearest-neighbour Spearman over the 6 matched trojan-gate pairs gives $\rho_s = 0.933$ (per-gate mean, ± 0.075 ; flat $\rho_s = 0.879$), again on a small n . The headline result, that the SHAP feature ranking the paper relies on (PO, FFi, LGFi, PI, FFi) is invariant to the 90 nm/180 nm distinction across the full test set, holds. Per-trojan node comparisons require more 90 nm trojan exemplars than Trust-Hub provides to draw stronger conclusions.

The 60/20/20 random split is reported as the primary evaluation result. Under this protocol, both train and test sets contain gates from all 30 circuits, and the model is assessed on its ability to distinguish trojan gates from benign gates given the same circuit-level feature distribution. The protocol is a controlled experimental condition for isolating XAI differences across methods, not a claim about realistic deployment. The LOCO and LOFO results are reported here to characterise the generalization boundary of the five-feature baseline.

Future research directions include multi-dataset validation across additional circuit families, cell libraries, and trojan insertion methods to test generalization beyond Trust-Hub; feature-space expansion incorporating functional simulation, timing analysis, and side-channel signatures, with domain-aware feature engineering (ratio properties such as LGFi/PO, interaction terms such as LGFi $\times$ FFi, and circuit-relative percentile features) to strengthen property-based explainability; advanced imbalance handling via synthetic oversampling and cost-sensitive learning; controlled human-subjects studies with practicing security engineers to quantify actionability and trust across XAI methods; hybrid explainability that combines domain-aware circuit analysis with model-agnostic attribution; extension to sequential-trigger trojan detection; and natural-language explanation synthesis using large language models [47, 48] to turn property patterns, feature attributions, and case exemplars into reports usable by engineers with varying expertise.

8 Conclusion

This work compares five explainability approaches for gate-level hardware trojan detection across two categories: domain-aware property analysis (M1) and model-agnostic methods, which split into precedent-driven case-based reasoning (M2) and feature attribution (LIME, SHAP, Gradient; M3–M5). Figure 4 and Table 2 summarize the taxonomy.

On 11,392 held-out test gates from Trust-Hub (157:1 benign-to-trojan), XGBoost at the val-optimal threshold reaches 48.08% precision and 69.44% recall (AUPRC = 0.637, 101× over no-skill), a 4.25-fold precision gain over the reimplemented Hasegawa SVM baseline (11.33% precision, 70.83% recall, F1 = 0.195). Random Forest reaches F1 = 0.555 at half the FP density (2.37 vs. 4.74 FP per 1,000 gates), confirming the gains are classifier-independent. McNemar’s test ($\chi^2(1) = 3578.23$, $p < 0.001$) and the 31.6 percentage-point accuracy gap (99.33% vs. 67.7%) confirm the difference between Method 1 and Methods 2–5 is statistically significant.

On explainability, property analysis produces 31 circuit-specific patterns (e.g., “high LGFi near PO indicates rare-event trigger circuits”); case-based reasoning achieves 96.51% correspondence between predictions and nearest training neighbors with full netlist provenance; LIME and SHAP agree at $\rho_s = 0.30$ per gate (95% CI [0.29, 0.31], $n = 11,392$) but yield generic importance scores; gradient attribution matches SHAP rankings at 7× the speed (0.16 ms vs. 1.10 ms per explanation). Method selection follows from these tradeoffs: property analysis where domain-aligned justifications aid engineer validation, case-based reasoning where precedent suffices, and feature attribution as exploratory tooling that still needs domain expertise to act on.

9 Declarations

9.1 Funding

The authors did not receive support from any organization for the submitted work.

9.2 Data Availability

Data and code used in this work are available at https://github.com/paulwhitten/expl_methods_hw_trojan_detection_code. The Trust-Hub benchmark dataset is publicly available at <https://www.trust-hub.org/>.

References

- [1] Ribeiro, M.T., Singh, S., Guestrin, C.: ” why should i trust you?” explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1135–1144 (2016)

- [2] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. *Advances in neural information processing systems* **30** (2017)
- [3] Simonyan, K., Vedaldi, A., Zisserman, A.: Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034* (2013)
- [4] Wolff, F., Papachristou, C., Bhunia, S., Chakraborty, R.S.: Towards trojan-free trusted ics: Problem analysis and detection scheme. In: *2008 Design, Automation and Test in Europe*, pp. 1362–1365 (2008). <https://doi.org/10.1109/DATE.2008.4484928>
- [5] Whitten, P., Wolff, F., Papachristou, C.: An ai architecture with the capability to classify and explain hardware trojans. In: *NAECON 2024 - IEEE National Aerospace and Electronics Conference*, pp. 349–354 (2024). <https://doi.org/10.1109/NAECON61878.2024.10670684> . <https://doi.org/10.1109/NAECON61878.2024.10670684>
- [6] Hasegawa, K., Oya, M., Yanagisawa, M., Togawa, N.: Hardware trojans classification for gate-level netlists based on machine learning. In: *2016 IEEE 22nd International Symposium on On-Line Testing and Robust System Design (IOLTS)*, pp. 203–206 (2016). <https://doi.org/10.1109/IOLTS.2016.7604700>
- [7] Hasegawa, K.: Hardware-trojan detection methods utilizing machine learning based on hardware-specific features. PhD thesis, Waseda University (2020)
- [8] Salmani, H., Tehranipoor, M., Sutikno, S., Wijitrisnanto, F.: Trust-Hub Trojan Benchmark for Hardware Trojan Detection Model Creation Using Machine Learning. <https://doi.org/10.21227/px6s-sm21> . <https://dx.doi.org/10.21227/px6s-sm21>
- [9] Negishi, R., Togawa, N.: Evaluation of ensemble learning models for hardware-trojan identification at gate-level netlists. In: *2024 IEEE International Conference on Consumer Electronics (ICCE)*, pp. 1–6 (2024). <https://doi.org/10.1109/ICCE59016.2024.10444240>
- [10] Salmani, H., Tehranipoor, M., Karri, R.: On design vulnerability analysis and trust benchmarks development. In: *2013 IEEE 31st International Conference on Computer Design (ICCD)*, pp. 471–474 (2013). <https://doi.org/10.1109/ICCD.2013.6657085>
- [11] Shakya, B., He, T., Salmani, H., Forte, D., Bhunia, S., Tehranipoor, M.: Benchmarking of hardware trojans and maliciously affected circuits. *Journal of Hardware and Systems Security* **1**, 85–102 (2017)
- [12] Slayback, S.M.: A computer scientist’s evaluation of publically available hardware trojan benchmarks. PhD thesis, Monterey, California: Naval Postgraduate School

(2015)

- [13] Rathor, V.S., Rastogi, A.: Ht-pred: An extensive methodology for dataset preparation and hardware trojan prediction using gate-level netlist. *Journal of Electronic Testing* **41**(4), 467–482 (2025)
- [14] King, G., Zeng, L.: Logistic regression in rare events data. *Political analysis* **9**(2), 137–163 (2001)
- [15] Mahfuz, T., Gaikwad, P., Suha, T., Bhunia, S., Chakraborty, P.: SALTy: Explainable Artificial Intelligence Guided Structural Analysis for Hardware Trojan Detection. In: 2025 IEEE 43rd VLSI Test Symposium (VTS), pp. 1–7 (2025). IEEE
- [16] Yasaei, R., Chen, L., Yu, S.-Y., Al Faruque, M.A.: Hardware trojan detection using graph neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* **44**(1), 25–38 (2022) <https://doi.org/10.1109/TCAD.2022.3178355>
- [17] Caruana, R., Kangaroo, H., Dionisio, J.D.N., Sinha, U.S., Johnson, D.B.: Case-based explanation of non-case-based learning methods. *Proceedings. AMIA Symposium*, 212–5 (1999)
- [18] Pan, Z., Mishra, P.: Hardware-assisted malware detection and localization using explainable machine learning. *IEEE Transactions on Computers* **71**(10), 2611–2622 (2022) <https://doi.org/10.1109/TC.2021.3120712>
- [19] Ma, P., Li, J., Liu, H., Shi, J., Zhang, S., Pan, W., Hao, Y.: Hardware Trojan detection methods for gate-level netlists based on graph neural networks. *IEEE Transactions on Computers* **74**(5), 1470–1481 (2025) <https://doi.org/10.1109/TC.2025.3533085>
- [20] Su, H., Hu, W., Zhang, X., Zhu, D., Wu, L.: Toward precise and explainable hardware Trojan localization at LUT level. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* **44**(7), 2817–2821 (2025) <https://doi.org/10.1109/TCAD.2025.3527377>
- [21] Gunning, D., Aha, D.W.: DARPA’s explainable artificial intelligence program. *AI Magazine* **40**(2), 44–58 (2019) <https://doi.org/10.1609/aimag.v40i2.2850>
- [22] Došilović, F.K., Brčić, M., Hlupić, N.: Explainable artificial intelligence: A survey. In: 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), pp. 0210–0215 (2018). <https://doi.org/10.23919/MIPRO.2018.8400040>
- [23] Fernandez, A., Herrera, F., Cordon, O., Jose del Jesus, M., Marcelloni, F.: Evolutionary fuzzy systems for explainable artificial intelligence: Why, when, what for,

- and where to? IEEE Computational Intelligence Magazine **14**(1), 69–81 (2019) <https://doi.org/10.1109/MCI.2018.2881645>
- [24] Hagras, H.: Toward human-understandable, explainable ai. Computer **51**(9), 28–36 (2018) <https://doi.org/10.1109/MC.2018.3620965>
 - [25] Howard, D., Edwards, M.A.: Explainable a.i.: The promise of genetic programming multi-run subtree encapsulation. In: 2018 International Conference on Machine Learning and Data Engineering (iCMLDE), pp. 158–159 (2018). <https://doi.org/10.1109/iCMLDE.2018.00037>
 - [26] Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D.: Grad-cam: Visual explanations from deep networks via gradient-based localization. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 618–626 (2017)
 - [27] Doshi-Velez, F., Kim, B.: Towards a rigorous science of interpretable machine learning. arXiv preprint arXiv:1702.08608 (2017)
 - [28] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., Pedreschi, D.: A survey of methods for explaining black box models. ACM computing surveys (CSUR) **51**(5), 1–42 (2018)
 - [29] Vilone, G., Longo, L.: Explainable artificial intelligence: a systematic review. arXiv preprint arXiv:2006.00093 (2020)
 - [30] Arrieta, A.B., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., *et al.*: Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information fusion **58**, 82–115 (2020)
 - [31] Vaughan, J., Sudjianto, A., Brahimi, E., Chen, J., Nair, V.N.: Explainable neural networks based on additive index models. arXiv preprint arXiv:1806.01933 (2018)
 - [32] Sweeney, J., Purdy, R., Blanton, R.D., Pileggi, L.: Circuitgraph: A python package for boolean circuits. Journal of Open Source Software **5**(56), 2646 (2020) <https://doi.org/10.21105/joss.02646>
 - [33] Shinan, E.: Lark: A modern parsing library for Python. PyPI (2024)
 - [34] Hagberg, A., Swart, P.J., Schult, D.A.: Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States) (2008)
 - [35] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery

- and Data Mining. KDD '16, pp. 785–794. Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2939672.2939785> . <https://doi.org/10.1145/2939672.2939785>
- [36] Whitten, P., Wolff, F., Papachristou, C.: An ai architecture with the capability to explain recognition results. In: 2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI), pp. 1–6 (2024). <https://doi.org/10.1109/ICMI60790.2024.10586116> . <https://doi.org/10.1109/ICMI60790.2024.10586116>
 - [37] Whitten, P., Wolff, F., Papachristou, C.: Case-Explainer: General-Purpose Case-Based Explainability, (2025). <https://github.com/paulwhitten/case-explainer>
 - [38] Whitten, P.C.: Explainable ai architectures: Methods, applications, examples, and results. PhD thesis, Case Western Reserve University (2025). http://rave.ohiolink.edu/etdc/view?acc_num=case1743462769378071
 - [39] Neyman, J., Pearson, E.S.: Ix. on the problem of the most efficient tests of statistical hypotheses. Philosophical Transactions of the Royal Society of London, Series A: Containing Papers of a Mathematical or Physical Character **231**(694-706), 289–337 (1933) <https://doi.org/10.1098/rsta.1933.0009> <https://royalsocietypublishing.org/rsta/article-pdf/231/694-706/289/240090/rsta.1933.0009.pdf>
 - [40] Green, D.M., Swets, J.A.: Signal Detection Theory and Psychophysics. Wiley, New York (1966)
 - [41] Fawcett, T.: An introduction to roc analysis. Pattern recognition letters **27**(8), 861–874 (2006)
 - [42] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
 - [43] Gansner, E.R., North, S.C.: An open graph visualization system and its applications to software engineering. Software: Practice and Experience **30**(11), 1203–1233 (2000)
 - [44] Japkowicz, N., Shah, M.: Evaluating learning algorithms: A classification perspective. Evaluating Learning Algorithms. A Classification Perspective (2011) <https://doi.org/10.1017/CBO9780511921803>
 - [45] Dietterich, T.G.: Approximate statistical tests for comparing supervised classification learning algorithms. Neural computation **10**(7), 1895–1923 (1998)
 - [46] Mohseni, S., Zarei, N., Ragan, E.D.: A multidisciplinary survey and framework for design and evaluation of explainable ai systems. ACM Transactions on Interactive

Intelligent Systems (TiiS) **11**(3–4), 1–45 (2021)

- [47] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., *et al.*: Language models are few-shot learners. *Advances in neural information processing systems* **33**, 1877–1901 (2020)
- [48] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., *et al.*: Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023)